

Java Backend Architecture

Interview Series

Chapter 7.8

Gradle Plugins

50+ Interview Q&A

Code Examples

Architecture Diagrams

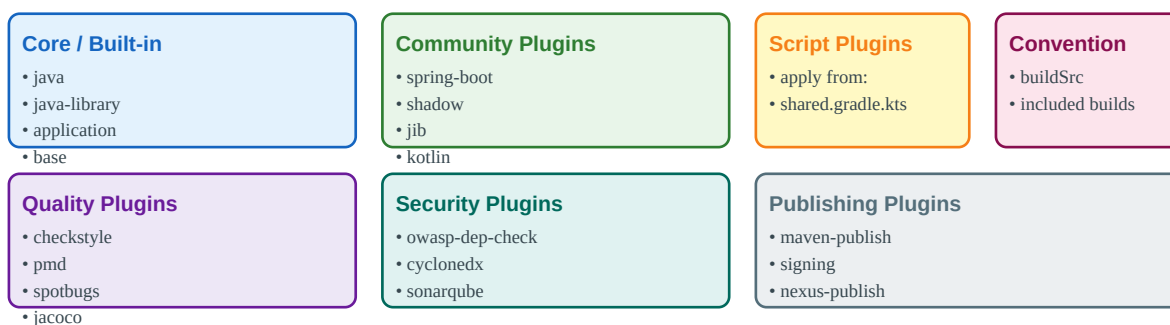
Advanced Topics

Chapter 7.8 — Gradle Plugins

Introduction

Gradle plugins extend the build system by adding tasks, configurations, conventions, and capabilities. They range from core built-in plugins (java, java-library, application) to community plugins (Spring Boot, Shadow, Jib) to custom plugins for organisation-specific conventions. Understanding how to apply, configure, and create plugins is essential for enterprise Gradle builds.

Plugin Categories



Applying Plugins

```
// Recommended: plugins {} block (type-safe, managed) plugins { java // built-in: shorthand
`java-library` // built-in: backtick syntax application // built-in
id("org.springframework.boot") version "3.2.0" id("io.spring.dependency-management") version
"1.1.4" id("com.github.johnrengelman.shadow") version "8.1.1" id("com.google.cloud.tools.jib")
version "3.4.0" } // apply false: declare in root but do NOT apply yet // (apply selectively in
subprojects) plugins { id("org.springframework.boot") version "3.2.0" apply false } // In
subproject: plugins { id("org.springframework.boot") // no version – inherited from root }
```

Java Plugin & Application Plugin

```
// java plugin (base for all JVM builds) plugins { java } java { sourceCompatibility =
JavaVersion.VERSION_17 targetCompatibility = JavaVersion.VERSION_17 // Preferred: toolchain
approach toolchain { languageVersion = JavaLanguageVersion.of(17) } withSourcesJar() //
includes -sources.jar in build outputs withJavadocJar() // includes -javadoc.jar in build
outputs } // application plugin (extends java) plugins { application } application { mainClass
= "com.example.MainApplication" applicationName = "my-app" executableDir = "bin" } //
application plugin adds tasks: // run – executes the application // installDist – creates
runnable installation in build/install // distTar / distZip – creates distribution archives
```

Spring Boot Plugin

bootJar

Executable fat JAR
(BOOT-INF layout)

bootWar

Executable WAR
with embedded container

bootRun

Run app locally
with live reload

build-image

OCI image via
Buildpacks (no Docker)

bootBuildImage

Docker image via
Spring Boot Buildpacks

```
plugins { id("org.springframework.boot") version "3.2.0" id("io.spring.dependency-management")
version "1.1.4" java } springBoot { mainClass = "com.example.Application" buildInfo() //
creates META-INF/build-info.properties (git info, etc.) } tasks.bootJar { archiveFileName =
"app.jar" // Layered JAR for efficient Docker builds layered { enabled = true application {
intoLayer("spring-boot-loader") { include("org/springframework/boot/loader/**") }
intoLayer("snapshot-dependencies") { include("*:*:SNAPSHOT") } intoLayer("dependencies")
intoLayer("application") } layerOrder("spring-boot-loader", "snapshot-dependencies",
"dependencies", "application") } } tasks.bootRun { jvmArgs("-Xmx512m",
"-Dspring.profiles.active=dev") }
```

Shadow Plugin (Uber-JAR)

```
plugins { id("com.github.johnrengelman.shadow") version "8.1.1" application } tasks.shadowJar
{ archiveClassifier = "all" mergeServiceFiles() // merge META-INF/services for SPI // Relocate
to avoid classpath conflicts relocate("com.google.common", "shadow.com.google.common") //
Exclude specific files exclude("META-INF/*.SF", "META-INF/*.DSA", "META-INF/*.RSA") // Include
only specific dependencies dependencies { exclude(dependency("org.slf4j:.*")) } }
```

Jib Plugin (Container Image)

```
plugins { id("com.google.cloud.tools.jib") version "3.4.0" } jib { from { image =
"eclipse-temurin:17-jre" platforms { platform { os = "linux"; architecture = "amd64" } platform
{ os = "linux"; architecture = "arm64" } } } to { image = "gcr.io/my-project/my-app" tags =
setOf("latest", project.version.toString()) auth { username = System.getenv("GCR_USERNAME")
password = System.getenv("GCR_PASSWORD") } } container { mainClass = "com.example.Application"
ports = listOf("8080") jvmFlags = listOf("-Xmx512m", "-XX:+UseG1GC") labels = mapOf("version"
to project.version.toString()) } } # Build and push image (no Docker daemon required) ./gradlew
jib # Build to local Docker daemon ./gradlew jibDockerBuild # Build to tarball ./gradlew
jibBuildTar
```

OWASP Dependency Check Plugin

```
plugins { id("org.owasp.dependencycheck") version "9.0.7" } dependencyCheck { failBuildOnCVSS =
7.0f // fail on HIGH severity (CVSS >= 7) formats = listOf("HTML", "SARIF", "JSON") nvd {
apiKey = System.getenv("NVD_API_KEY") } // Suppress false positives suppressionFile =
"dependency-check-suppression.xml" } # Run vulnerability scan ./gradlew dependencyCheckAnalyze
./gradlew dependencyCheckAggregate # multi-module aggregate report
```

Quality Plugins: Checkstyle, PMD, SpotBugs

```
plugins { checkstyle pmd id("com.github.spotbugs") version "6.0.6" } checkstyle { toolVersion =
"10.12.7" configFile = file("config/checkstyle/checkstyle.xml") // Google Style
isIgnoreFailures = false } pmd { toolVersion = "7.0.0" ruleSets =
listOf("category/java/bestpractices.xml", "category/java/errorprone.xml") isIgnoreFailures =
false } spotbugs { toolVersion = "4.8.3" effort = com.github.spotbugs.snom.Effort.MAX
reportLevel = com.github.spotbugs.snom.Confidence.LOW excludeFilter =
file("config/spotbugs-exclusions.xml") } tasks.withType<com.github.spotbugs.snom.SpotBugsTask>
{ reports.create("html") { enabled = true } }
```

Creating a Custom Plugin

```
// buildSrc/src/main/kotlin/com/example/MyPlugin.kt
import org.gradle.api.Plugin
import org.gradle.api.Project
class MyPlugin : Plugin<Project> {
    override fun apply(project: Project) {
        // Add an extension (DSL block)
        val extension = project.extensions.create("myConfig",
            MyExtension::class.java)
        // Register a task
        project.tasks.register("generateDocs",
            GenerateDocsTask::class.java) {
            title.set(extension.title)
        }
        // React to other plugins being applied
        project.plugins.withType<JavaPlugin> {
            // Configure Java-specific behaviour
        }
    }
}
interface MyExtension {
    val title: Property<String>
}
// buildSrc/src/main/kotlin/com/example/GenerateDocsTask.kt
abstract class GenerateDocsTask
@Inject constructor() : DefaultTask() {
    @get:Input abstract val title: Property<String>
    @get:OutputFile abstract val outputFile: RegularFileProperty
    @TaskAction fun generate() {
        outputFile.get().asFile.writeText("# ${title.get()}")
    }
}
```

50+ Interview Questions & Answers

Q1. What is a Gradle plugin?

A reusable piece of build logic that adds tasks, configurations, conventions, and capabilities to a project. Plugins are applied using the `plugins {}` block or legacy `apply()` method.

Q2. What is the difference between the `plugins {}` block and `apply()`?

`plugins {}` is the modern declarative approach — resolved from the Gradle Plugin Portal, type-safe, and supports version management. `apply()` is legacy — used for script plugins or when dynamic application is needed.

Q3. What is `apply false` in the `plugins` block?

Declares a plugin version in the root build without applying it. Used to manage plugin versions centrally: subprojects can then apply the plugin without specifying the version.

Q4. What is the difference between the `java` plugin and `java-library` plugin?

`java` adds implementation, `testImplementation`, and related configurations. `java-library` adds the api configuration for exposing dependencies to consumers. Use `java-library` for libraries, `java` for applications.

Q5. What does the `application` plugin add?

Extends `java` with a `run` task (executes main class), `installDist` (creates installation directory), `distTar`/`distZip` (creates distribution archives). Requires `mainClass` property.

Q6. What is the Spring Boot Maven plugin equivalent in Gradle?

The `org.springframework.boot` plugin. `bootJar` creates an executable fat JAR, `bootWar` creates executable WAR, `bootRun` starts the app, `bootBuildImage` creates a container image using Buildpacks.

Q7. What is a layered JAR in Spring Boot?

A fat JAR structured into layers (spring-boot-loader, dependencies, snapshot-dependencies, application) optimised for Docker caching. Each layer corresponds to a Docker layer — unchanged layers are cached.

Q8. How does `bootBuildImage` differ from a Dockerfile?

`bootBuildImage` uses Cloud Native Buildpacks — no Dockerfile needed, no Docker daemon required (by default). It automatically detects the JVM type, applies security patches, and creates an OCI-compliant image.

Q9. What is the Shadow plugin?

Creates uber-JARs (fat JARs) for Gradle projects. Merges all dependencies into a single JAR, supports package relocation, service file merging, and dependency filtering.

Q10. What is the Jib plugin?

Google's containerisation plugin that builds OCI images without a Dockerfile or Docker daemon. Builds images layer-by-layer (application, dependencies, resources separately) for efficient layer caching.

Q11. What are the three Jib tasks?

jib — builds and pushes directly to a container registry. jibDockerBuild — builds to the local Docker daemon. jibBuildTar — builds to a local tarball.

Q12. Why use Jib over a Dockerfile for Java apps?

No Docker daemon required (works in CI without Docker-in-Docker). Reproducible builds. Automatic layer optimisation (deps layer is stable, only app code changes). Multi-platform image support.

Q13. What is the OWASP dependency-check plugin?

Scans project dependencies against the NVD CVE database to identify known vulnerabilities. Configurable CVSS score threshold for build failure. Generates HTML, SARIF, and JSON reports.

Q14. What is the failBuildOnCVSS setting?

Sets the minimum CVSS score to fail the build. 7.0 = HIGH severity, 9.0 = CRITICAL only. Recommended to fail on 7.0+ for production applications.

Q15. What does the checkstyle plugin do?

Runs Checkstyle static analysis against Java source code. Enforces code style rules (Google Style, Sun Checks, custom XML config). Fails the build on violations when isIgnoreFailures = false.

Q16. What does the PMD plugin check?

Runs PMD (best practices, error-prone patterns, unused code, empty catch blocks, etc.) and CPD (copy-paste detector for code duplication). Configurable rule sets and violation thresholds.

Q17. What is SpotBugs?

Bytecode analysis tool detecting potential bugs: null dereferences, infinite loops, resource leaks, SQL injection patterns, and more. Successor to FindBugs. More thorough than PMD but requires compiled code.

Q18. How do you suppress false positives in SpotBugs?

Create a spotbugs-exclusions.xml file defining patterns for bugs to ignore: by class, method, bug code, or bug category. Reference with excludeFilter in the spotbugs configuration.

Q19. What is the JaCoCo plugin?

Measures code coverage during test execution. jacocoTestReport generates HTML/XML/CSV coverage reports. jacocoTestCoverageVerification enforces minimum coverage thresholds and fails the build if not met.

Q20. What is the SonarQube plugin?

Sends build metrics (test results, coverage, code quality, security issues) to a SonarQube server for comprehensive code quality analysis and trend tracking. Run with ./gradlew sonarqube.

Q21. What is the maven-publish plugin?

Publishes artifacts to Maven repositories (local or remote). Configure publications (what to publish: JAR, sources, javadoc, POM) and repositories (where to publish: local, Nexus, Maven Central).

Q22. How do you publish to Maven Central with Gradle?

Apply maven-publish and signing plugins. Configure publication with components.java, withSourcesJar(), withJavadocJar(). Sign artifacts. Use nexus-publish or publishing.repositories with OSSRH URL and credentials.

Q23. What is the signing plugin used for?

Signs artifacts using GPG before publication. Required for Maven Central. Configure: `signing { sign(publishing.publications['mavenJava']) }`. Reads GPG key from `gradle.properties` or environment.

Q24. What is a plugin extension?

A configuration DSL block added by a plugin: `jib { ... }`, `springBoot { ... }`. Extensions are created with `project.extensions.create()`. They provide type-safe plugin configuration as an alternative to task properties.

Q25. How do you implement a Gradle plugin?

Implement `Plugin` interface, override `apply(project)`. Add extensions for configuration, register tasks, configure conventions. Deploy as a JAR in `buildSrc`, an included build, or the Gradle Plugin Portal.

Q26. What is the Gradle Plugin Portal?

`plugins.gradle.org` — the public repository for community Gradle plugins. Plugins declared in `plugins {}` are resolved from here by default (or from `pluginManagement` repositories).

Q27. What is `pluginManagement` in `settings.gradle.kts`?

Configures where plugin JARs are resolved from and allows specifying plugin versions centrally. Also supports `resolutionStrategy` for mapping plugin IDs to specific versions.

Q28. What is the CycloneDX plugin?

Generates a Software Bill of Materials (SBOM) in CycloneDX format. Lists all project dependencies with their coordinates, licenses, and known vulnerabilities. Required for supply chain security compliance.

Q29. What is the `nexus-publish` plugin?

Simplifies publishing to Sonatype OSSRH (Nexus) for Maven Central. Provides `closeAndReleaseStagingRepository` task that handles the staging, closing, and releasing workflow required by Central.

Q30. How do you configure a plugin version in version catalogs?

In `libs.versions.toml`: `[plugins] spring-boot = { id = 'org.springframework.boot', version.ref = 'spring-boot' }`. Apply in `build.gradle.kts`: `alias(libs.plugins.spring.boot)`.